Procedurálne programovanie

slido slido.com
#7867887
PrPr 04 (26)

Ján Zelenka

Ústav Informatiky

Slovenská akadémia vied



Obsah prednášky

slido slido.com
#7867887
PrPr 04 (26)

1. **Opakovanie – Typová konverzia**
2. **Ukazovateľ**
3. **Reťazce**

Spätná väzba: *https://forms.gle/5ft8RFuLtVb4gRXm8*

Bodové hodnotenie – Projek (35%)

1. verzia projektu

- odovzdanie v 6. týždni (23.10. do 23:59)
  - neskoré odovzdanie je penalizované => ***uznáva sa iba 60% zo získaných bodov***

IDstudenta_projekt_v1.c

2. verzia projektu

- odovzdanie v 8. týždni (6.11. do 23:59)
  - neskoré odovzdanie je penalizované => ***uznáva sa iba 60% zo získaných bodov***

IDstudenta_projekt_v2.c

3. verzia projektu

- odovzdanie v 11. týždni (29.11. do 23:59)

IDstudenta_projekt_v3.c

Obhajoba projektu (povinná časť, 0 hodnotenie obhajoby = neuznávanjú sa body projektu)

Typová konverzia

slido slido.com
#7867887
PrPr 04 (26)

Typový systém

- každá hodnota je na najnižšej úrovni reprezentovaná ako postupnosť bitov
- každá hodnota má počas výpočtu priradený typ
- typ hodnoty dáva postupnosti bitov význam – ako sa má interpretovať

Sú pravidlá:

- pre zmenu typu
- definujúce ktorý typ môže byť použitý danou operáciou

C je statický typový systém

- typ je kontrolovaný počas prekladu

Typová konverzia

- prevod premennej určitého typu na iný typ
  - napr. `int` na `float`

- dva druhy konverzií:
  - implicitná - samovoľná, automatická
  - explicitná - vynútená, požadovaná

Implicitná typová konverzia

Pred vykonaním operácie sa samostatné operandy konvertujú:

- kedykoľvek sa objaví typ **`char`** alebo **`short int`**, konvertuje sa na **`int`**
- všetky operandy **`unsigned char`** a **`unsigned short`** sa konvertujú na **`int`** - ak **`int`** nepretečie, inak na **`unsigned int`**

Implicitná typová konverzia

Ak majú dva operandy jednej operácie rôzny typ, operand s nižšou prioritou je konvertovaný na typ s vyššou prioritou podľa hierarchie:

```
int            ⇒ unsigned int
unsigned int   ⇒ long
long           ⇒ unsigned long
unsigned long  ⇒ float
float          ⇒ double
double         ⇒ long double
```

- `int` má najnižšiu prioritu
- jeden operand typu `float` a druhý nižšiu prioritu, druhý ⇒ `float`

Implicitná typová konverzia

V priraďovacích príkazoch je typ na pravej strane konvertovaný na typ z ľavej strany.

```
double x;

x = 5;
```

x bude mať hodnotu 5.0

```
int i;

i = 5.0;
```

i bude mať hodnotu 5

Explicitná typová konverzia

Pretypovanie

- jazyk C dovoľuje takmer ľubovoľnú konverziu → riziko, že to bude nevhodné

Syntax: `(typ) výraz`

Príklady:

`(int) char_vyraz`	- prevod znaku na ordinálne číslo
`(char) int_vyraz`	- prevod ordinálneho čísla na znak
`(int) double_vyraz`	- odrezanie desatinnej časti
`(double) int_vyraz`	- prevod celého čísla na reálne
`(double) float_vyraz`	- zväčšenie presnosti

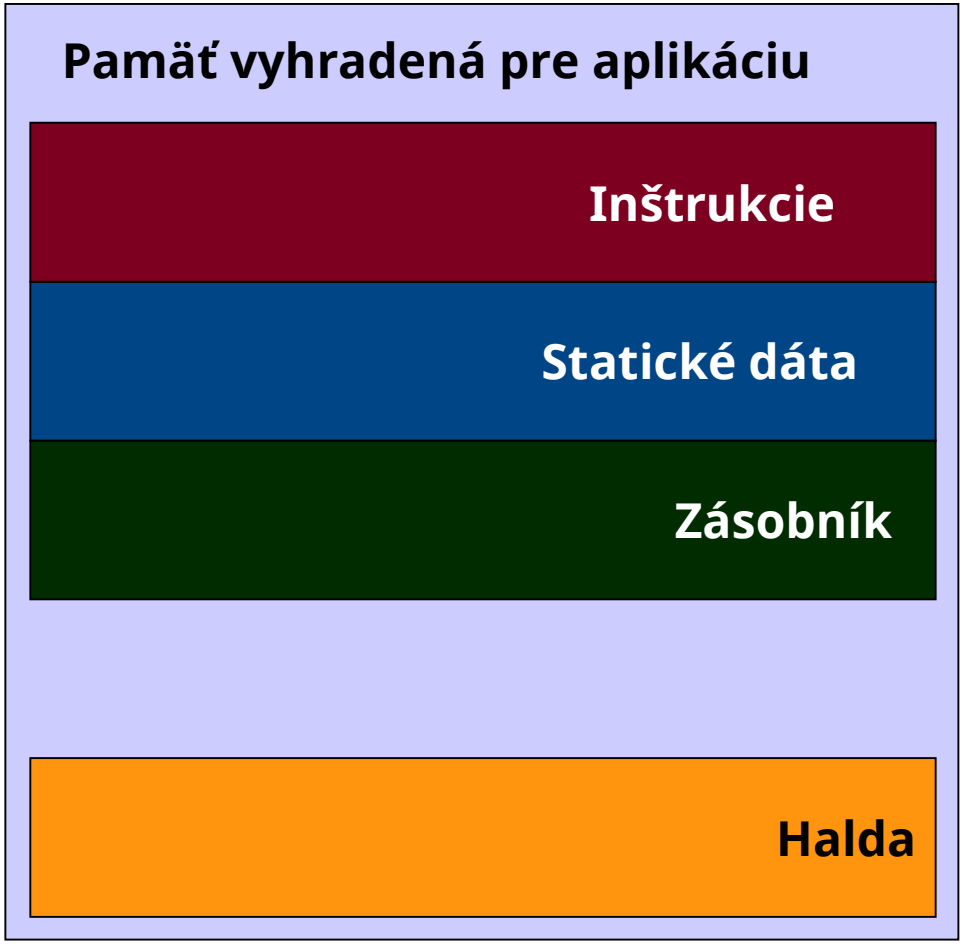
Ukazovatele

Pamäť

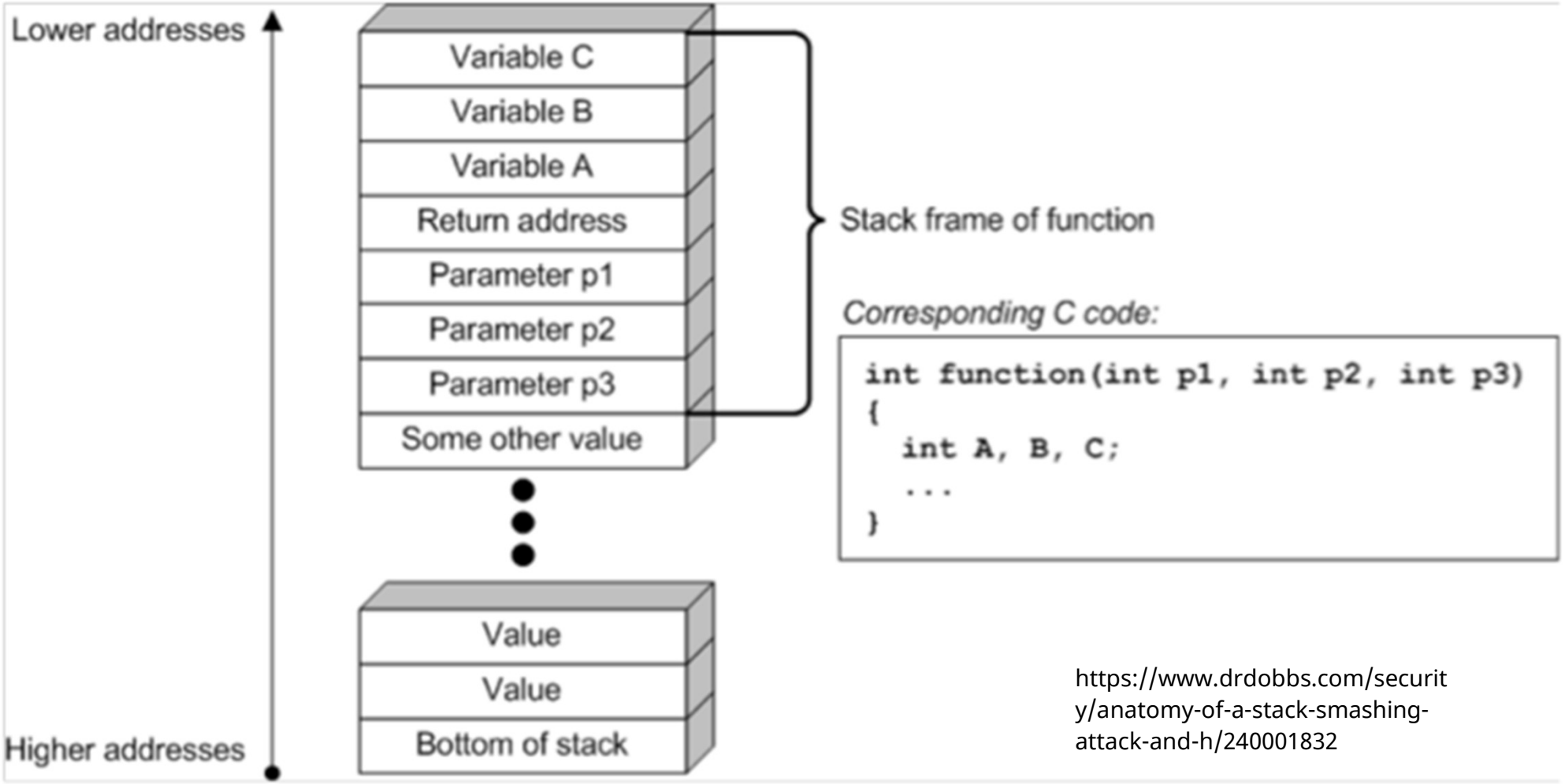
- pamäť obsahuje veľké množstvo buniek s pevnou veľkosťou
  - adresované zvyčajne na úrovni 8 bitov (1 bajt)
- v pamäti sú umiestnené entity (premenné, reťazce…)
- entita zaberá viacej ako jednu bunku
  - napr. premenná typu int zaberá 4 bajty
  - adresa entity v pamäti je prvá bunka kde je entita umiestnená
- adresy sú uvádzané v hexadecimálnej sústave (s predponou 0x)

Organizácia pamäte

- Inštrukcie
  - Asemblerovský kód aplikácie
- Statické dáta (static)
  - Globálne premenné
- Zásobník (stack)
  - Volania funkcií
    (iné pre každú funkciu)
  - Lokálne premenné
- Halda, hromada (heap)
  - Dynamicky alokované premenné
    (malloc, free)
  - Iné pri každom spustení



Zásobník



https://www.drdobbs.com/security/anatomy-of-a-stack-smashing-attack-and-h/240001832

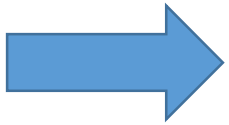
Adresa premennej
Referenčný operátor &

slido slido.com #7867887 PrPr 04 (26)

- Operátor **&** vracia adresu svojho argumentu
  - miesto v pamäti, kde je argument uložený
- Adresa (jeho výstup) sa priraďuje do ukazovateľa

```
int i = -10;
int *p_i = &i;
```

int i = -10;
(i je meno miesta v pamäti s adresou X, kde je uložená hodnota -10)



Obsah pamäťovej bunky	**Adresa** pamäťovej bunky
-10	X

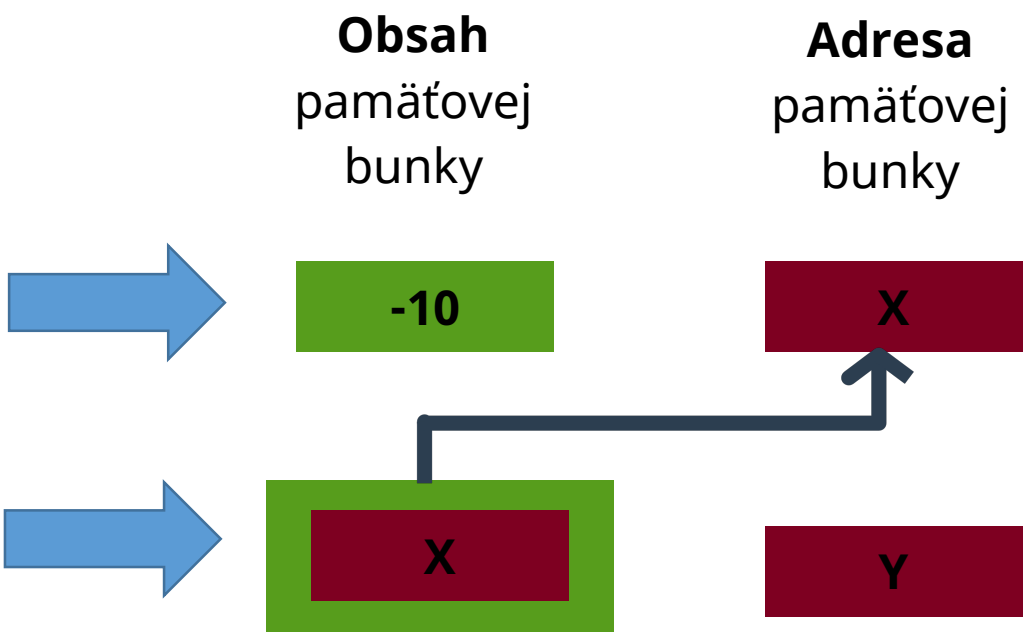
Adresa premennej
Referenčný operátor &

- Operátor `&` vracia adresu svojho argumentu
  - miesto v pamäti, kde je argument uložený
- Adresa (jeho výstup) sa priraďuje do ukazovateľa

```
int i = -10;
int *p_i = &i;
```

`int i = -10;`
(i je meno miesta v pamäti s adresou X, kde je uložená hodnota -10)

`int *p_i = &i;`
(p_i je meno miesta v pamäti s adresou Y, kde je uložená adresa X)



Premenná VS ukazovateľ

Štandardná premenná

- pomenovanie pamäťové miesto, ktoré má svoju adresu X

Ukazovateľ

- všetky ukazovatele majú rovnakú veľkosť (sú to pamäťové miesta na uchovanie adresy - dátový typ ukazovateľa je adresa)
- jeho typ musí zodpovedať typu premennej, na ktorú ukazuje
  - ako sa majú bity na adrese Y interpretovať?
- ak ukazovateľ ukazuje na určitú adresu, vieme že na tejto adrese začína a ďalej spojito pokračuje cez toľko bajtov, koľko je potrebných na uloženie premennej príslušného typu.

Definícia ukazovateľa

- špecifikujeme typ ukazovateľa
- pred menom premennej je *

```
int i;
int *p_i;
```

je ekvivalentné

```
int i, *p_i;
```

iba premenná **a** je ukazovateľ

```
int *a, b;
```

- Pozor na neinicializovaný ukazovateľ
  - Ukazuje na náhodné miesto v pamäti
  - Pred prvým použitím je potrebná jeho inicializácia

Inicializácia ukazovateľa

Priradíme premennej p adresu premennej i.

```
int i, *p_i;
...
p_i = &i;
```

Ukazovateľ p_i ukazuje na premennú i.

Ukazovateľ, ktorý nikam neukazuje

- Nulový ukazovateľ: **`NULL`** neobsahuje žiadnu platnú adresu – neukazuje na žiadne konkrétne miesto v pamäti, s ktorým chceme pracovať.
- **`NULL`** - symbolická konštanta definovaná v **`stdio.h`**:
  - **`#define NULL 0`**
  - **`#define NULL ((void *) 0)`**
- Je možné priradiť ho ukazovateľom na ľubovoľný typ

```
int *p;
p = NULL;
```

```
if (p == NULL)
...
```

Údaje na adrese
Dereferenčný operátor *

- Umožňuje pracovať s obsahom pamäťového miesta na ktoré ukazuje ukazovateľ
  - Pozor nie vlastnú hodnotu ukazovateľa – to je adresa
  - Ukazovateľ môže ukazovať na hodnotu, ktorá je interpretovaná ako adresa -> **`int**`**

```
int i = 7;
int *p = &i;
*p = 10;
```

- obsah pamäte, na ktorú ukazuje **`p`** sa zmení na 10
- **`p`** ukazuje na to isté miesto v pamäti

```
p = 10;
```

- **CHYBA**: meníme miesto, na ktoré ukazuje **`p`**

Ukazovateľ na ukazovateľ

```
int a, *p_a, **p_p;

p_a = &a;
p_p = &p_a;

int b = *p_a;
int c = **p_p;

 *p_p = p_a;
**p_p = 10;
 *p_p = &a;
```

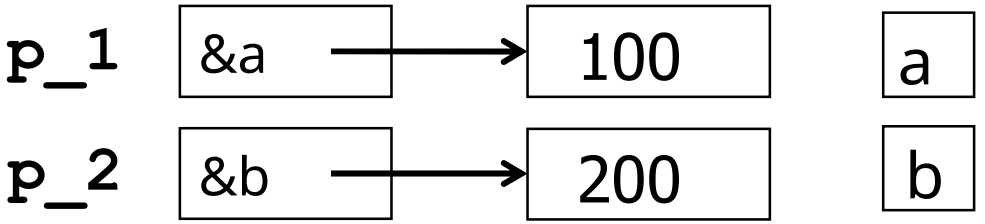
```
a == *p_a == **p_p
&a == p_a == *p_p
&p_a == p_p
```

Základné operácie s ukazovateľmi

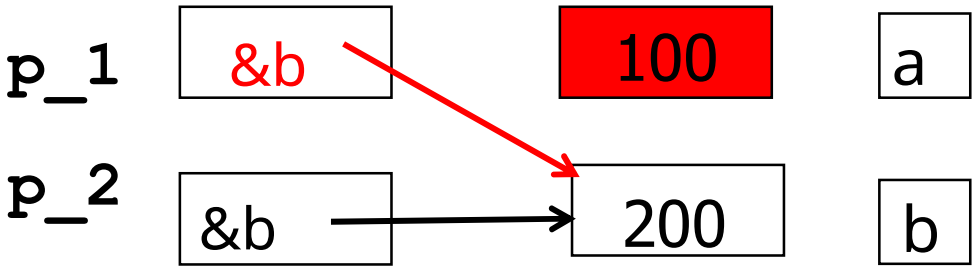
```
int *p_1, *p_2;
...
p_1 = p_2;
```

```
int a = 100, b = 200;
p_1 = &a;
p_2 = &b;
```

pred príkazom `p_1 = p_2;`



po príkaze `p_1 = p_2;`



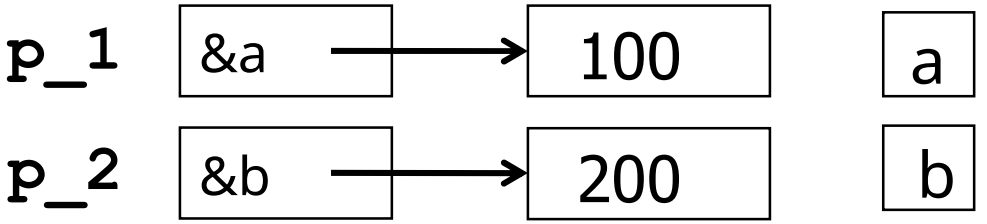
p_1 ukazuje na rovnakú premennú ako p_2

Základné operácie s ukazovateľmi

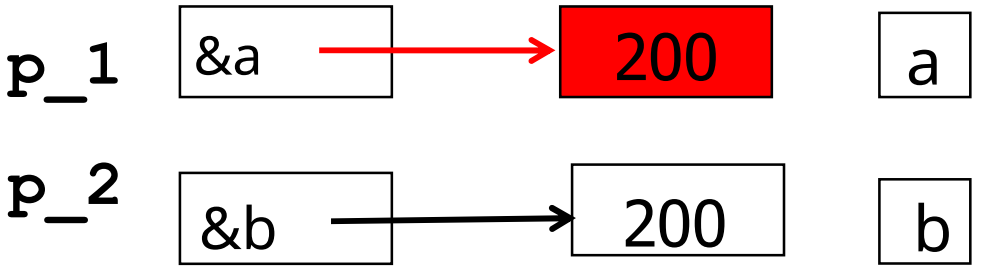
```
int *p_1, *p_2;
...
*p_1 = *p_2;
```

```
int a = 100, b = 200;
p_1 = &a;
p_2 = &b;
```

pred príkazom `*p_1 = *p_2;`



po príkaze `*p_1 = *p_2;`



Nemení sa hodnota ukazovateľa,
ale hodnota premennej na ktorú `p_1` ukazuje.

Základné operácie s ukazovateľmi

```
int i, *p_i;
```

```
p_i = &i;
```
- správne

```
p_i = &(i + 3);
```
- chyba: `(i + 3)` nie je premenná

```
p_i = &15;
```
- chyba: konštanta nemá adresu

```
p_i = 15;
```
- chyba: priraďovanie absolútnej adresy

```
i = p_i;
```
- chyba: priraďovanie adresy

```
i = & p_i;
```
- chyba: priraďovanie adresy

```
*p_i = 4;
```
- správne, ak `p_i` bol inicializovaný

Výpis adresy

Špecifikácia formátu (v `printf()`): %p

```
int i, *p_i;

p_i = &i;
printf("Adresa i: %p, hodnota p_i:
%p\n", &i, p_i);
```

Segmentation fault

Majme ukazovateľ na celočíselnú premennú

```
int i, *p_i;
...
p_i = &i;
```

Priraďme mu adresu, ktoré nie je prístupná programu

```
p_i = 10;
```

Pri pokuse o prístup mimo vyhradenú pamäť

```
*p_i = 7;
```

Segmentation fault

Viacej ukazovateľov na rovnaké miesto v pamäti

```
int i = 1;
int *p_1 = &i;
int *p_2 = &i;
int *p_3 = &i;
```

Všetky ukazovatele môžu meniť dané miesto v pamäti

```
*p_3 = 5;
*p_1 = 62;
*p_2 = 39;
```

Pozor na vzájomné prepisovanie si hodnôt

Konverzia ukazovateľov

- Vyhnúť sa jej!
- Ak sa nedá vyhnúť – explicitne pretypovávať

```
int *p_i;
char *p_c;

p_c = p_i;
p_c = (char *)p_i;
```

nevhodné

vhodnejšie

Ukazovateľ typu `void`

- V čase definície ukazovateľa nie je zrejmé na aký typ premennej bude ukazovať.
- Použijeme generický ukazovateľ

```
void *p;
```

- Môže ukazovať na ľubovoľný typ

- Pred konkrétnym použitím generického ukazovateľa je potrebné **pretypovanie** na typ konkrétnej premennej, na ktorú bude ukazovať.

Ukazovateľ typu `void`

Pri priraďovaní je potrebné uviesť typ

```
int i;
float f;
void *p_void = &i;

*(int *) p_void = 2;
p_void = &f;
*(float *) p_void = 3.5;
```

`p_void` ukazuje na `i`

nastavenie `i` na 2

`p_void` ukazuje na `f`

nastavenie `f` na 3.5

Parametre funkcií
Odovzdanie hodnotou

Odovzdávanie hodnotou

```
void foo(int X){
    X = 3;
    // v X je 3
}
int main(){
    int variable = 0;
    foo(variable);
    //v X je znova 0
    return 0;
}
```

- Hodnota sa nakopíruje do novej lokálnej premennej
- Po zániku lokálnej premennej sa zmena nepropaguje mimo tela funkcie

Parameter je ukazovateľ

```
void foo(int *P){
    *P = 3;
}
int main(){
    int X = 0;
    foo(&X);
    // v X je 3
    return 0;
}
```

- Riešením je predávať (ako hodnotu) adresu premennej
- Modifikuje sa hodnota na tejto adrese, namiesto lokálnej premennej
- Takto sa odovzdávajú premenné, ktoré sa majú meniť

Parametre funkcií
Odovzdanie odkazom

- predávanie parametrov odkazom v čistom C neexistuje (podporuje ho C++)
  - volanie odkazom by umožnilo meniť parametre v rámci funkcie tak, aby zostali aj po jej skončení
  - rieši sa pomocou ukazovateľov
  - ukazovateľ určuje, na ktorom mieste vo volajúcej funkcii sa má premenná zmeniť (nemení sa ukazovateľ - adresa)

Ukazovateľ na funkciu

- Udalosťami riadené programovanie (Event-driven)
  - napr. zaregistrovanie funkcie ako (callback) reakcie pri nejakej udalosti (napr. antivírové programy)
- Ukazovateľ na funkciu obsahuje adresu umiestnenia kódu funkcie
  - Namiesto hodnoty je na adrese kód funkcie
- Ukazovateľ na funkciu môže byť parameter inej funkcie
  - Predáme miesto, kde je zápis toho, čo sa má vykonať

```
void foo(int i,int(*p_f)(float, float));
```

Ukazovateľ na funkciu

```
#include <stdio.h>
void vypis(int a)
{
    printf("Zadana hodnota je %d\n", a);
}

int main()
{
    void (*p_vypis)(int) = &vypis;

    (*p_vypis)(10);

    return 0;
}
```

`void (*p_vypis)(int);`
`p_vypis = &vypis;`

`p_vypis` je ukazovateľ na funkciu vypis()

Zavolanie funkcie pomocou ukazovateľa

- Meno funkcie sa dá použiť na získanie jej adresy
- operátor & nie je povinný
- vždy bez zátvoriek

- Ukazovateľ na funkciu sa dá zavolať ako funkcia
- Operátor * nie je povinný

Ukazovateľ na funkciu

Dôležitá je signatúra funkcie

- typ a počet argumentov, typ návratovej hodnoty
- meno funkcie nie je dôležité
- do ukazovateľa na funkciu s danou signatúrou môžeme priradiť adresy všetkých funkcií s **rovnakou signatúrou**

```
int Scitaj (int a, int b){…}
int Vynasob (int a, int b){…}
int main (void) {
    int (*p_f) (int, int) = &Scitaj;
    p_f(1,3);
    p_f = &Vynasob;
    p_f(7,2);
    return 0;
}
```

Príklady definícií

```
int i;
```
- **i** je typu **int**

```
float *y;
```
- **y** je ukazovateľ na typ **float**

```
double *z();
```
- **z** je funkcia vracajúca ukazovateľ na **double**

```
int (*v)();
```
- ukazovateľ na funkciu vracajúcu **int**

```
int *(*v)();
```
- ukazovateľ na funkciu vracajúcu ukazovateľ na **int**

Príklad - v1

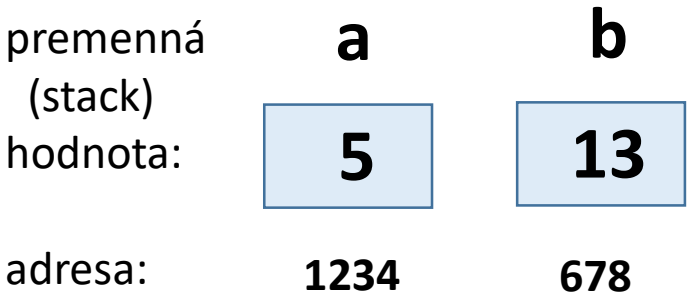
```
void vymen(int *p_a, int *p_b)
{
 int pom;
 pom = *p_a;
 *p_a = *p_b;
 *p_b = pom;
}

int main (void) {
 int a=5, b=13;
 printf("a: %3d, b: %3d\n",a,b);
 vymen(&a, &b);
 printf("a: %3d, b: %3d\n",a,b);
 return 0;
}
```

Príklad - v1

```
void vymen(int *p_a, int *p_b)
{
 int pom;
 pom = *p_a;
 *p_a = *p_b;
 *p_b = pom;
}

int main (void) {
 int a=5, b=13;
 printf("a: %3d, b: %3d\n",a,b);
 vymen(&a, &b);
 printf("a: %3d, b: %3d\n",a,b);
 return 0;
}
```



Príklad - v1

```
void vymen(int *p_a, int *p_b)
{
 int pom;
 pom = *p_a;
 *p_a = *p_b;
 *p_b = pom;
}

int main (void) {
 int a=5, b=13;
 printf("a: %3d, b: %3d\n",a,b);
 vymen(&a, &b);
 printf("a: %3d, b: %3d\n",a,b);
 return 0;
}
```

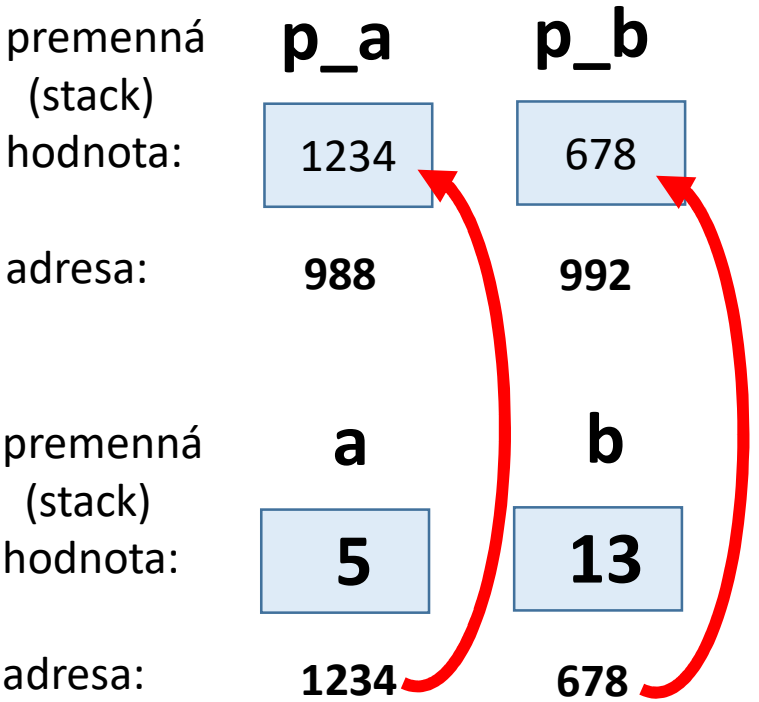
premenná (stack)	a	b
hodnota:	5	13
adresa:	1234	678

```
a:   5, b:  13
```

Príklad - v1

```
void vymen(int *p_a, int *p_b)
{
 int pom;
 pom = *p_a;
 *p_a = *p_b;
 *p_b = pom;
}

int main (void) {
 int a=5, b=13;
 printf("a: %3d, b: %3d\n",a,b);
 vymen(&a, &b);
 printf("a: %3d, b: %3d\n",a,b);
 return 0;
}
```



Príklad - v1

```
void vymen(int *p_a, int *p_b)
{
 int pom;
 pom = *p_a;
 *p_a = *p_b;
 *p_b = pom;
}

int main (void) {
 int a=5, b=13;
 printf("a: %3d, b: %3d\n",a,b);
 vymen(&a, &b);
 printf("a: %3d, b: %3d\n",a,b);
 return 0;
}
```

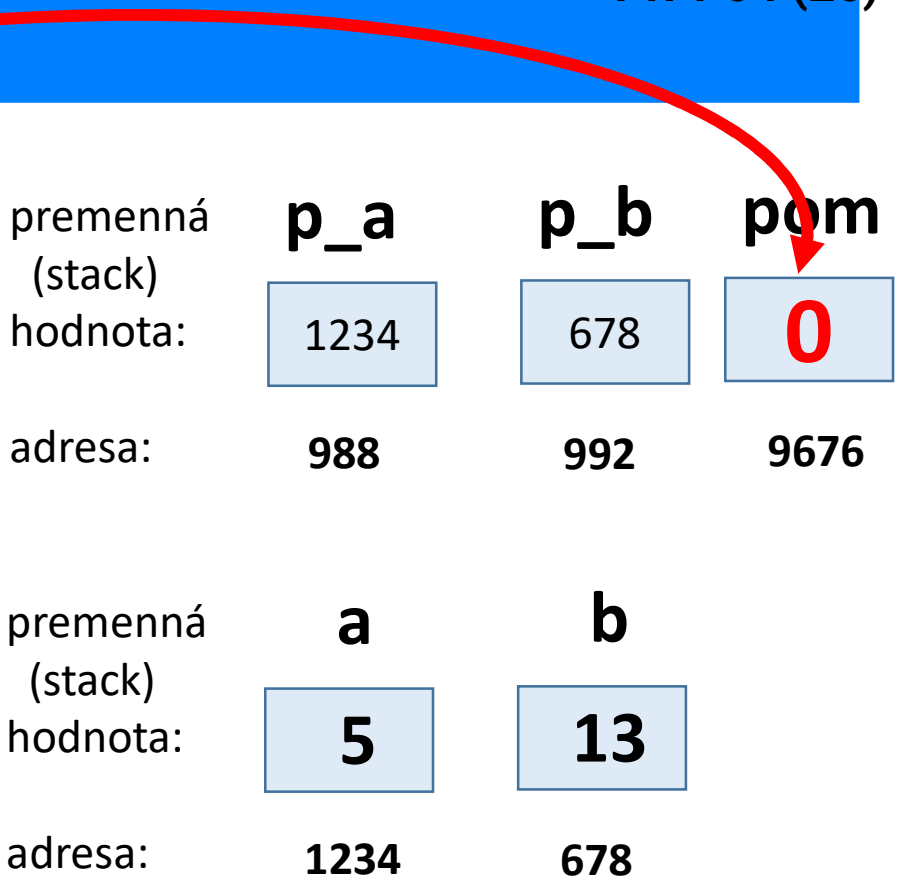
premenná (stack)	p_a	p_b	pom
hodnota:	1234	678	54324
adresa:	988	992	9676

premenná (stack)	a	b
hodnota:	5	13
adresa:	1234	678

Príklad - v1

```
void vymen(int *p_a, int *p_b)
{
 int pom = 0;
 pom = *p_a;
 *p_a = *p_b;
 *p_b = pom;
}

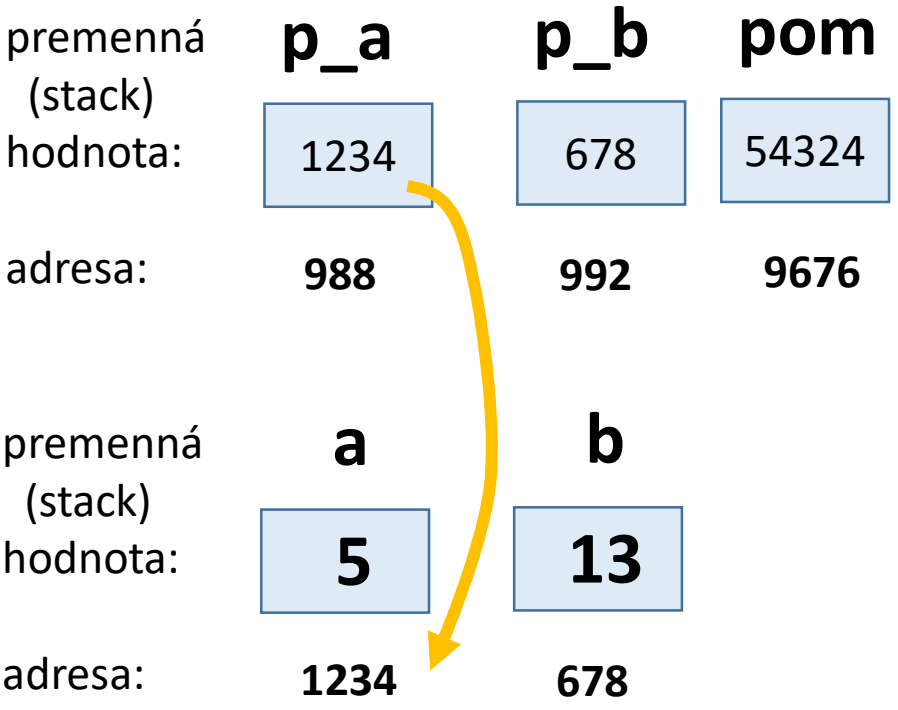
int main (void) {
 int a=5, b=13;
 printf("a: %3d, b: %3d\n",a,b);
 vymen(&a, &b);
 printf("a: %3d, b: %3d\n",a,b);
 return 0;
}
```



Príklad - v1

```
void vymen(int *p_a, int *p_b)
{
 int pom;
 pom = *p_a;
 *p_a = *p_b;
 *p_b = pom;
}

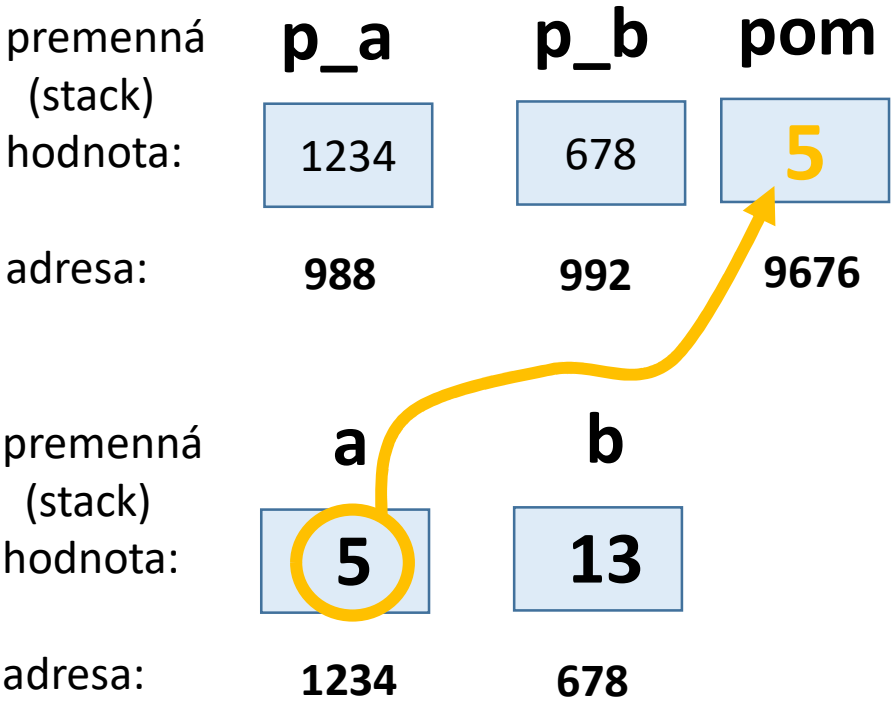
int main (void) {
 int a=5, b=13;
 printf("a: %3d, b: %3d\n",a,b);
 vymen(&a, &b);
 printf("a: %3d, b: %3d\n",a,b);
 return 0;
}
```



Príklad - v1

```
void vymen(int *p_a, int *p_b)
{
 int pom;
 pom = *p_a;
 *p_a = *p_b;
 *p_b = pom;
}

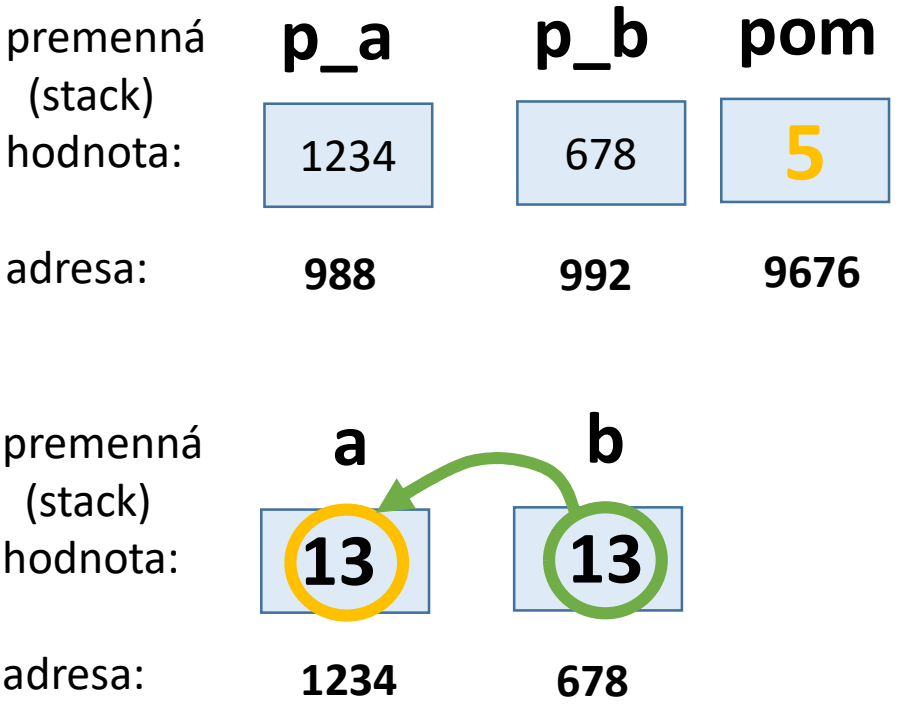
int main (void) {
 int a=5, b=13;
 printf("a: %3d, b: %3d\n",a,b);
 vymen(&a, &b);
 printf("a: %3d, b: %3d\n",a,b);
 return 0;
}
```



Príklad - v1

```
void vymen(int *p_a, int *p_b)
{
 int pom;
 pom = *p_a;
 *p_a = *p_b;
 *p_b = pom;
}

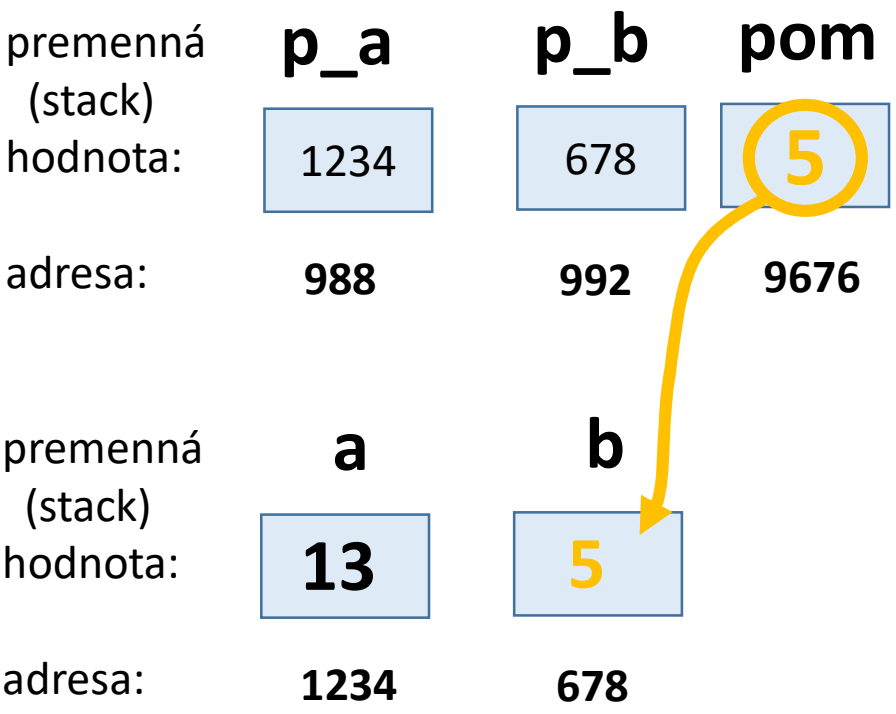
int main (void) {
 int a=5, b=13;
 printf("a: %3d, b: %3d\n",a,b);
 vymen(&a, &b);
 printf("a: %3d, b: %3d\n",a,b);
 return 0;
}
```



Príklad - v1

```
void vymen(int *p_a, int *p_b)
{
 int pom;
 pom = *p_a;
 *p_a = *p_b;
 *p_b = pom;
}

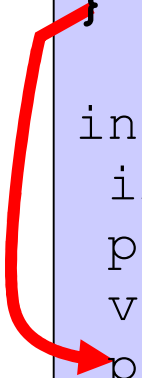
int main (void) {
 int a=5, b=13;
 printf("a: %3d, b: %3d\n",a,b);
 vymen(&a, &b);
 printf("a: %3d, b: %3d\n",a,b);
 return 0;
}
```

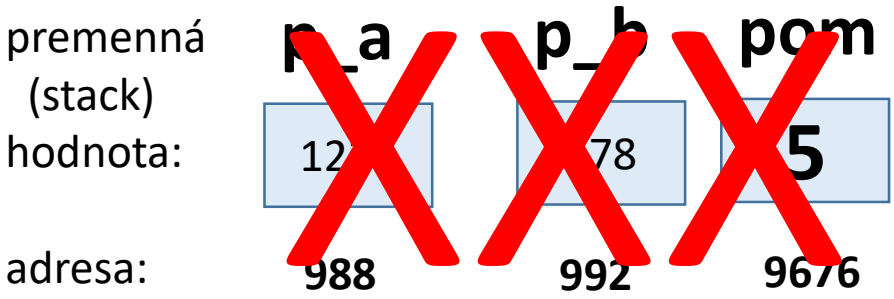


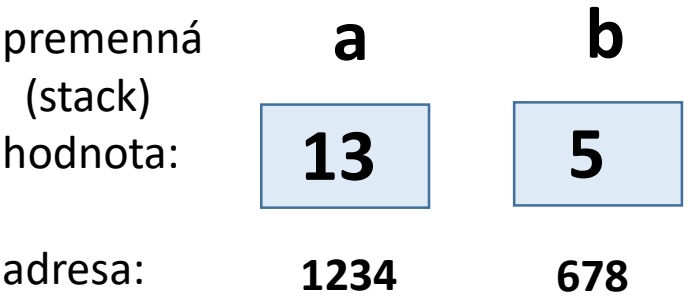
Príklad - v1

```
void vymen(int *p_a, int *p_b)
{
 int pom;
 pom = *p_a;
 *p_a = *p_b;
 *p_b = pom;
}

int main (void) {
 int a=5, b=13;
 printf("a: %3d, b: %3d\n",a,b);
 vymen(&a, &b);
 printf("a: %3d, b: %3d\n",a,b);
 return 0;
}
```



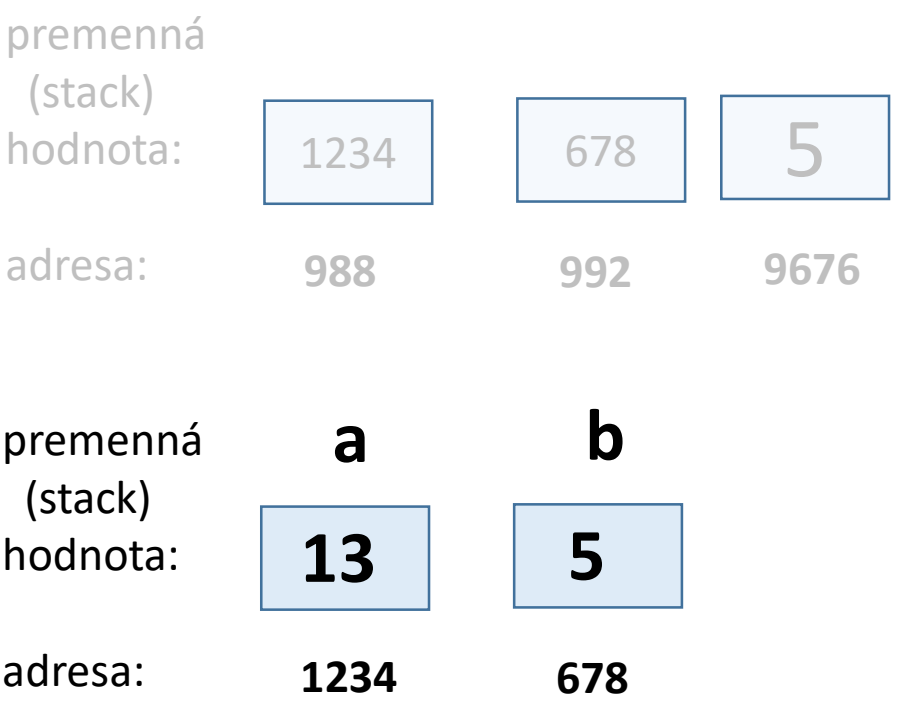




Príklad - v1

```
void vymen(int *p_a, int *p_b)
{
 int pom;
 pom = *p_a;
 *p_a = *p_b;
 *p_b = pom;
}

int main (void) {
 int a=5, b=13;
 printf("a: %3d, b: %3d\n",a,b);
 vymen(&a, &b);
 printf("a: %3d, b: %3d\n",a,b);
 return 0;
}
```

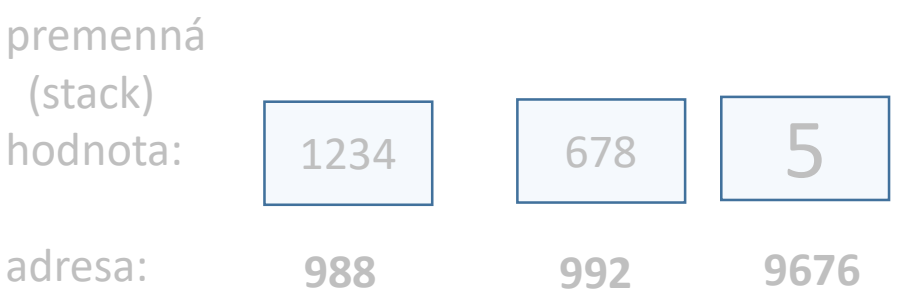


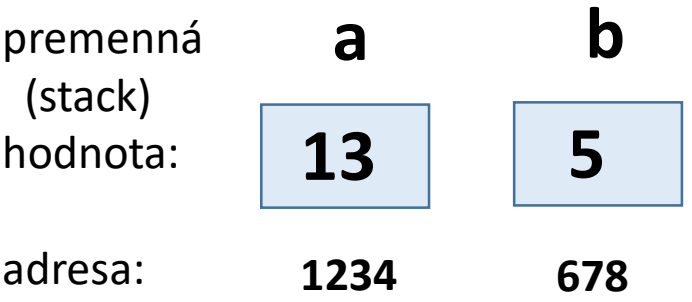

```
a:   5, b:  13
a:  13, b:   5
```

Príklad - v1

```
void vymen(int *p_a, int *p_b)
{
 int pom;
 pom = *p_a;
 *p_a = *p_b;
 *p_b = pom;
}

int main (void) {
 int a=5, b=13;
 printf("a: %3d, b: %3d\n",a,b);
 vymen(&a, &b);
 printf("a: %3d, b: %3d\n",a,b);
 return 0;
}
```





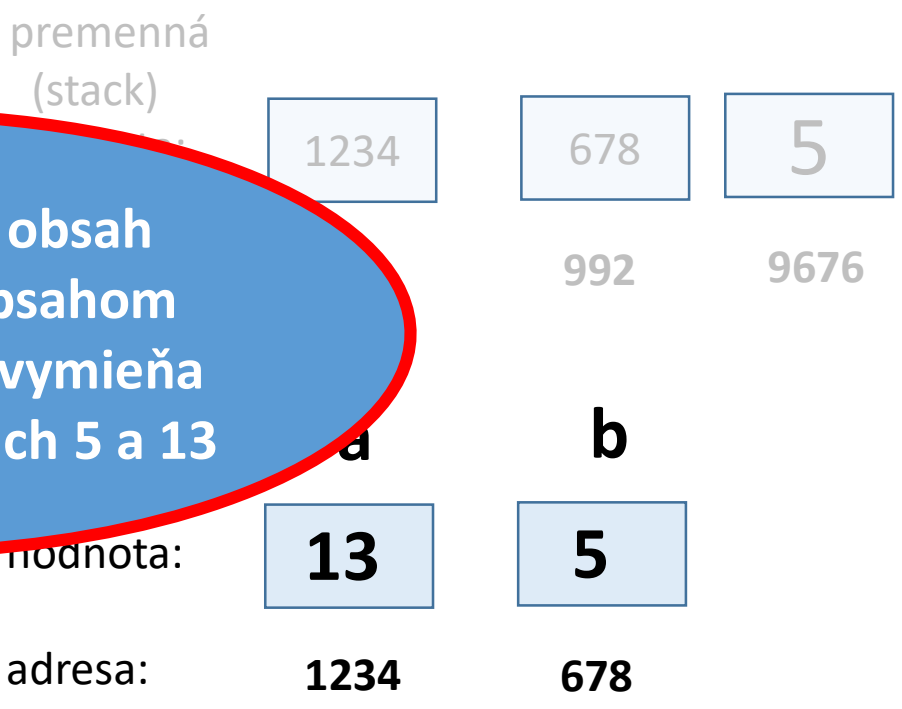

```
a:   5, b:  13
a:  13, b:   5
```

Príklad - v1

```
void vymen(int *p_a, int *p_b)
{
 int pom;
 pom = *p_a;
 *p_a = *p_b;
 *p_b = pom;
}

int main (void) {
 int a=5, b=13;
 printf("a: %3d, b: %3d\n",a,b);
 vymen(a, b);
 printf("a: %3d, b: %3d\n",a,b);
 return 0;
}
```

chyba: vymieňa obsah adries, daných obsahom premenných a, b: vymieňa hodnoty na adresách 5 a 13



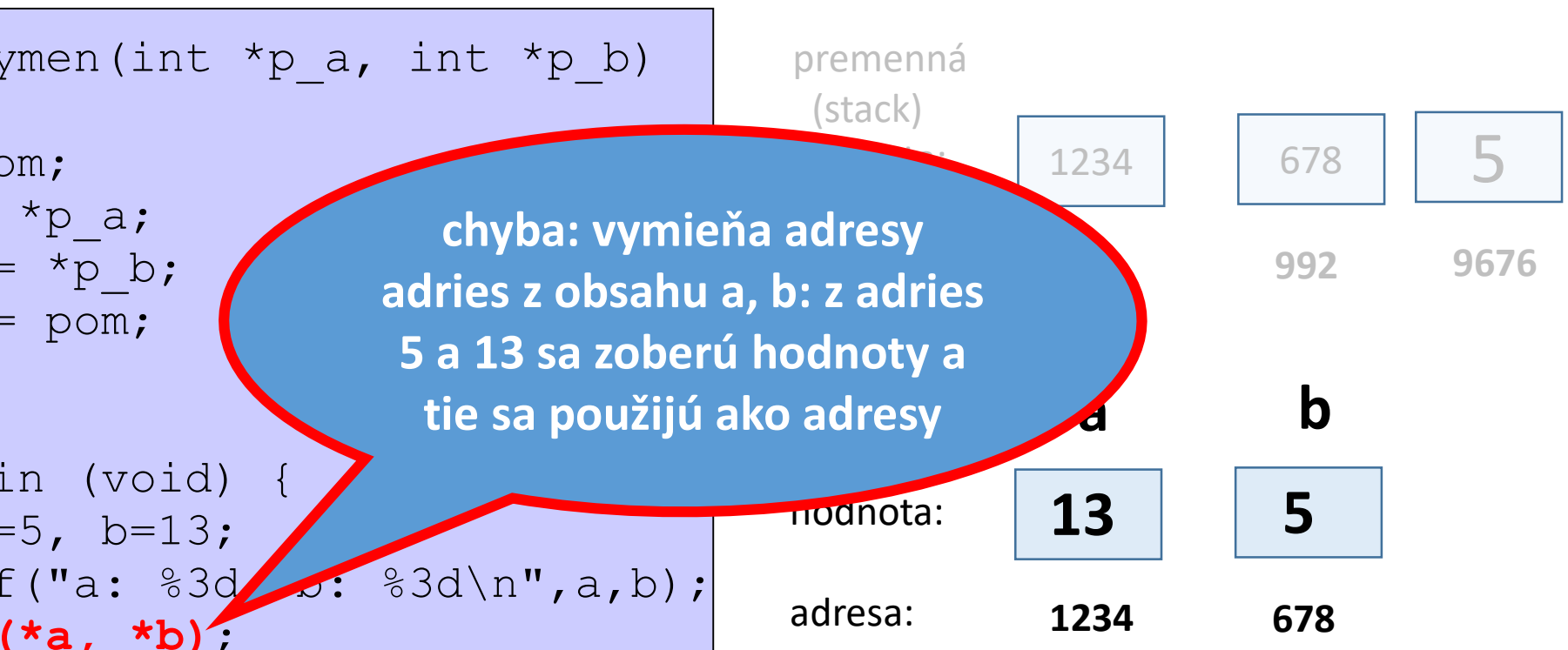

```
a:   5, b:  13
a:  13, b:   5
```

Príklad - v1

```
void vymen(int *p_a, int *p_b)
{
 int pom;
 pom = *p_a;
 *p_a = *p_b;
 *p_b = pom;
}

int main (void) {
 int a=5, b=13;
 printf("a: %3d, b: %3d\n",a,b);
 vymen(*a, *b);
 printf("a: %3d, b: %3d\n",a,b);
 return 0;
}
```

chyba: vymieňa adresy adries z obsahu a, b: z adries 5 a 13 sa zoberú hodnoty a tie sa použijú ako adresy




```
a:   5, b:  13
a:  13, b:   5
```

Príklad – v2

```
#include <stdio.h>
#define PI 3.14
#define na_druhu(i) ((i) * (i))
void kruh(  int r,    float *o,   float *s)
{
   *o = 2 * PI * r;
   *s = PI * r* r ;    //*s = PI * na_druhu(r);
}
int main()
{
    int polomer;
    float obvod, obsah;
     printf("Zadaj polomer kruhu: ");
     scanf(" %d", &polomer);
     kruh(polomer, &obvod, &obsah);
     printf("obvod: %.2f, obsah: %.2f\n", obvod, obsah);
     return 0;
```

Príklad – v3

```
void vymen1( char**p_x,  char**p_y)
{
   char *p;
   p = *p_x;
   *p_x = *p_y;
   *p_y = p;
}
```

```
char   c = ’a’,   *p_c  =  &c,
       d = ‘b’,   *p_d =  &d;

vymen1(  &p_c,  &p_d);
```

Príklad – v3

```
void vymen2( void **p_x,  void **p_y)
{
   void *p;
   p = *p_x;
   *p_x = *p_y;
   *p_y = p;
}
```

```
char   c = ' a ',   *p_c  =  &c,
       d = ' b ',   *p_d =  &d;

vymen2( (void **)&p_c, (void **)&p_d);
```

Príklad – v3

```
void vymen2( void **p_x,  void **p_y)
{
   void *p;
   p = *p_x;
   *p_x = *p_y;
   *p_y = p;
}
```

```
char   c = ' a ',   *p_c  =  &c,
       d = ' b ',   *p_d =  &d;
int    e = 1,       *p_e = &e,
       f = 13,      *p_f = &f;

vymen2( (void **)&p_c, (void **)&p_d);
vymen2( (void **)&p_e, (void **)&p_f);
```

Reťazce

slido slido.com
#7867887
PrPr 04 (26)

Reťazce

- znaková premenná uchováva kód znaku
  - celé číslo
- reťazce sú jednorozmerné **polia typu char**
  - z celkovej pamäte je aktívna len časť od začiatku poľa po znak '\0' ukončovací znak
  - ak nie je reťazec ukončený znakom '\0', považuje sa za reťazec celá nasledujúca oblasť v pamäti až do najbližšieho znaku '\0'
  - dĺžka reťazca je pozícia znaku '\0'
    - nemusíme si ju pamätať v premennej
    - skrátenie pomocou posunu null

Reťazce

reťazec s najviac **5 znakmi**:

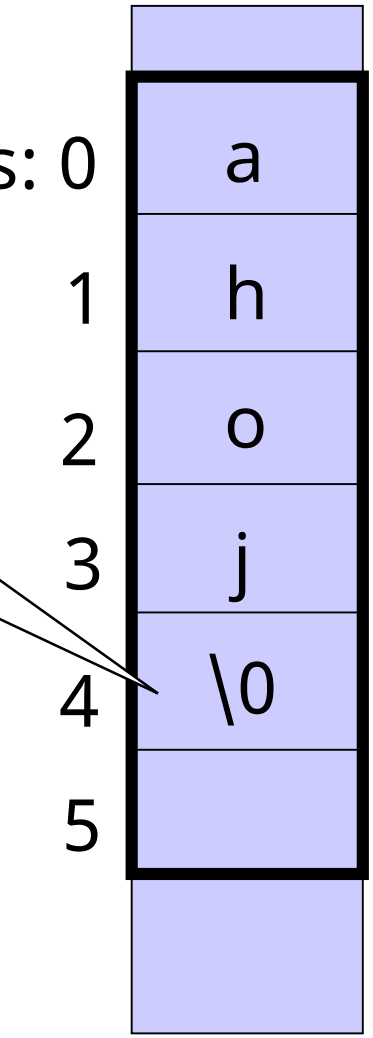
- staticky:

```
char s[6] = "ahoj";
```

```
char s[] = "abrakadabra";
```

- "dynamicky":

```
char *s;
s = (char *) malloc(6);
```



znak null alebo 0 (ASCII kód 0) nepliesť si s cifrou 0 (jej kód je 48)

inicializuje sa miesto práve pre daný text

Reťazce

```
char s[] =
{'A','h','o','j'};
```

neukončený '\0'

```
char s[] =
{'A','h','o','j',0};
```

ukončený '\0'

```
char s[6] = "ahoj";
```

pole inicializované na "ahoj"

```
char *s = "ahoj";
```

s nie je dynamický reťazec, ale ukazovateľ na typ **char** a je inicializovaný adresou reťazcovej konštanty (ktorá má obsah ahoj)

Reťazce

```
char *retazec = "Jan";
retazec = "Karol";
retazec[3] = 's';
```

nie je možné meniť reťazcovú konštantu

```
char s[10] = "Hallo";
s = "ahoj";
s[3] = 'r';
```

nie je možné priradiť statickému reťazcu konštantu

Reťazce

- 'z' vs "retazcova_konstanta"
- statická časť programu
- Inicializácia reťazca (== pole znakov ukončené '\0'):
- ako pole:
```
char s[] = {'A','h','o','j','\0'};
```
- pomocou konštanty:
```
char s[] = "Ahoj";
```

Reťazce

- Pracujeme s ním ako s jednorozmerným poľom
  - operátor []

```
char s[10];

for (i = 0; i < 10-1; i++)
    s[i] = '*';
s[10-1] = '\0';
```

dôležité: ukončiť reťazec!

Reťazce

- Môžeme využiť ukazateľovú aritmetiku

```
char s[] = "Ahoj svet";

*(s + 4) = 0; //s[4]

Char *s2 = s + 5;
```

s je adresa na prvý znak reťazca

zmena konca reťazca

ukazovateľ na časť iného reťazca

- Reťazce nemenia svoju veľkosť automaticky
  - automatické zväčšovanie
  - zápis za koniec poľa
- operátor + sčíta ukazovatele na reťazce nie je to zreťazenia (Java, C++)

Reťazce

```
char s[10];
...
scanf(" %s", s);
```

sem nepartrí **&**, pretože **s** je adresa

- **scanf()** vynecháva biele znaky a číta po prvý biely znak
  - Výhoda nemusíme ošetrovať prázdne znaky na začiatku riadku
  - Nevýhoda načítavanie sa ukončí pri prvom prázdnom znaku
- ak je na vstupe **" ahoj Eva!"**, **scanf()** prečíta iba **"ahoj"** a zvyšok zostáva v bufferi klávesnice

Formátovaný vstup a výstup z a do reťazca

- použitie výhod formátovaného vstupu a výstupu, ale nevypísať nič na obrazovku ani nenačítavať

```
int sprintf(char *s, char *format, ...);
```

pracuje ako **fprintf**, ale zapisuje do reťazca **s**

```
int sscanf(char *s, char *format, ...);
```

pracuje ako **fscanf**, ale číta z reťazca **s**

Riadkovo orientovaný vstup a výstup z terminálu

- okrem **`scanf()`**, **`printf()`** aj:

```
char *gets(char *s);
```

číta celý riadok do **`s`**: na koniec nezapíše **`\n`**, ale **`\0`**, vracia ukazovateľ na **`s`**, ak je riadok prázdny dáva do **`s`** **`\0`** a vráti **`NULL`**

```
int puts(char *s);
```

vypíše reťazec a odriadkuje (**`\n`**), vráti nezáporné číslo ak sa podarilo vypísať, inak **`EOF`**

Riadkovo orientovaný vstup a výstup z terminálu

- okrem `fscanf()`, `fprintf()` aj:

```
char *fgets(char *s, int max, FILE *fr);
```

číta riadok zo súboru do konca riadku ale maximálne **`max`** znakov, načítané zapíše do **`s`** (aj s **`\n`**), vracia ukazovateľ na **`s`**, ak je koniec súboru tak **`NULL`**

```
int fputs(char *s, FILE *fw);
```

do súboru **`fw`** vypíše reťazec **`s`**, neodriadkuje ani neukončuje pomocou **`\0`**, vráti nezáporné číslo ak sa podarilo vypísať, inak **`EOF`**

Reťazce

Definícia typu pre reťazce

```
typedef char *STRING;
```

Treba rozlišovať:

- nulový ukazovateľ **`NULL`** a nulový reťazec `'\0'`:
  - nulový ukazovateľ neukazuje na žiadne miesto v pamäti,
  - nulový reťazec má v 0-tom prvku znak `'\0'`
- **`"x"`** a **`'x'`**:
  - **`"x"`** je reťazec s jedným znakom ukončený `'\0'` (2 Byty)
  - **`'x'`** je jeden znak (1 Byte)

Štandardné funkcie pre prácu s reťazcami

- Reťazec sa nedá kopírovať priradením
- Reťazce sa nedajú porovnávať (==, !=, >, < atď.)

Funkcionality si treba naprogramovať, alebo použiť funkcie z knižnice

- nie sú súčasťou samotného jazyka C

- Využitie knižnice **string.h**
  - dokumentácia http://www.cplusplus.com/reference/cstring/
  - prehľad: https://en.cppreference.com/w/c/string/byte
- Základné pravidlá
  - reťazec je ukončený znakom '\0'
  - pri modifikácii reťazca má výstupný reťazec dostatočnú veľkosť

Štandardné funkcie pre prácu s reťazcami

```
int strlen(char *s);
```

vracia dĺžku reťazca (bez `\0`)

```
char *strcpy(char *kam, char *co);
```

kopírovanie reťazca **`co`** do **`kam`**, vracia ukazovateľ na **`kam`**
(reťazec **`kam`** musí byť dosť dlhý)

Štandardné funkcie pre prácu s reťazcami

```
char *strcat(char *kam, char *co);
```

pripojí reťazec **co** ku **kam**, vracia ukazovateľ na **kam**
(reťazec **kam** musí byť dosť dlhý
-> strlen(kam) + strlen(co) + 1)

```
int strcmp(char *s1, char *s2);
```

vracia **0**, ak sú reťazce rovnaké, záporné číslo, ak **s1** je skôr (abecedne), inak kladné číslo
(porovnáva sa na základe kódov znakov -> 'Z' je skôr ako 'a'

Štandardné funkcie pre prácu s reťazcami

```
char *strchr(char *s, char c);
```

nájdenie znaku **c** v reťazci **s**, vracia prvý výskyt znaku, ak sa v **s** nenachádza, vráti **NULL**

```
char *strstr(char *s1, char *s2);
```

vracia ukazovateľ na prvý výskyt reťazca **s2** v reťazci **s1**, v prípade neúspechu **NULL**

Práca s časťou reťazca

- podobne ako uvedené funkcie,
- v názve je **`n`** (zo slova *number*), napr. **`strncpy()`**:

```
char *strncpy(char *s1, char *s2, int max);
```

kopírovanie najviac max znakov z reťazca **`s2`** do **`s1`**, vracia ukazovateľ na **`s1`**

Práca s reťazcom naopak

- podobne ako uvedené funkcie,
- v názve je r (zo slova *reverse*), napr. **`strrchr()`**:

```
char *strrchr(char *s, char c);
```

nájdenie znaku **`c`** v reťazci **`s`**, vracia posledný výskyt znaku, ak sa v **`s`** nenachádza, vráti NULL

Prevody reťazcov na čísla

- konvertovanie reťazca číslic na číslo (funkcie definované v **`stdlib.h`**)

```
int atoi(char *s);
```

prekonvertuje reťazec znakov na **`int`**

```
long atol(char *s);
```

prekonvertuje reťazec znakov na **`long`**

```
float atof(char *s);
```

prekonvertuje reťazec znakov na **`float`**

Pri vstupe a výstupe nie je konverzia potrebná (**`scanf()`** a **`printf()`**)

Kontrola znakov v reťazci

Knižnica **type.h** – http://www.cplusplus.com/reference/cctype/

- **isalnum(znak)** – je znak písmeno alebo číslica?
- **isdigit(znak)** – je znak desiatková číslica?
- **isxdigit(znak)** – je znak hexadecimálna číslica?
- **isalpha(znak)** – je znak písmeno (veľké/malé)?
- **islower(znak)** – je znak malé písmeno?
- **isupper(znak)** – je znak veľké písmeno?
- **ispunct(znak)** – je znak špeciálny (vypísateľný)?
- **isprint(znak)** – dá sa znak vypísať (medzera, písmeno...)?
- **isgraph(znak)** – má znak grafickú podobu (písmeno, číslica…)?
- **isspace(znak)** – je znak biely (nový riadok, medzera..)?
- **iscntrl(znak)** – je znak riadiaci?

Prevod znakov:

- **tolower(znak)** – veľké písmeno prevedie na malé písmeno
- **toupper(znak)** – malé písmeno prevedie na veľké písmeno

Pole reťazcov

```
char *p_text[4], **p;
...
p = p_text;
puts(*++p);
```

vypíše sa **"druhy"** pretože sme najprv zvýšili **p** o 1 (posunuli sme ho na druhý riadok a potom vypísali reťazec, kam ukazuje **p**)

```
char *p_text[4], **p;
...
p = p_text;
for (i = 0; i < 4; i++)
    puts(*p++);
```

++ má väčšiu prioritu ako ***,** riadok sa najprv vypíše a ukazovateľ **p** sa posunie na druhý riadok.

Časté chyby pri práci s reťazcami

- nedostatočne veľký cieľový reťazec (napr. **`strcpy()`**)
- nekorektné ukončenie reťazca
  - funkcie z knižnice string sa správajú nekorektne
  - napr. zapisujeme mimo rozsah poľa
  - vzniká napr. pri prepísaní, nevložením... znaku `'\0'`
- dĺžka/veľkosť reťazca sa udáva bez znaku `'\0'` (**`strlen()`**)
- operátor `==` neporovnáva reťazce (**`strcmp()`**)
- operátor `+` nezreťazuje reťazce

Ďakujem vám za pozornosť!

Spätná väzba: *https://forms.gle/5ft8RFuLtVb4gRXm8*